



Asenkron JS & Web Workers

JavaScript'i Daha Hızlı ve Etkili Kullanmak

JS

Senkron vs Asenkron JavaScript

Callback

Basitçe bir fonksiyon içerisinde başka bir fonksiyonun çağırılması işlemi

```
const stepOne = (call_two) => {  
  console.log('stepOne');  
  call_two();  
}  
  
const stepTwo = () => {  
  console.log('stepTwo');  
}  
  
stepOne(stepTwo);
```

Senkron vs Asenkron JavaScript

Callback Hell Problemi

```
setTimeout(() => {  
  console.log("1. Adım Tamamlandı");  
  setTimeout(() => {  
    console.log("2. Adım Tamamlandı");  
    setTimeout(() => {  
      console.log("3. Adım Tamamlandı");  
    }, 1000);  
  }, 1000);  
}, 1000);
```

```
const process = (step) => {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log(`${step} Tamamlandı`);  
      resolve();  
    }, 1000);  
  });  
};  
  
process("1. Adım")  
  .then(() => process("2. Adım"))  
  .then(() => process("3. Adım"));
```



Senkron vs Asenkron JavaScript

- JavaScript'in Tek Thread'li Çalışma Mantığı
- Bloklayan (Senkron) ve Bloklamayan (Asenkron) İşlemler.

```
console.log("Birinci işlem");  
setTimeout(() => console.log("İkinci işlem (gecikmeli)"), 2000);  
console.log("Üçüncü işlem");
```

output

Birinci işlem

Üçüncü işlem

İkinci işlem (gecikmeli)

Senkron vs Asenkron JavaScript

async/await ile Daha Temiz Kod

```
const getData = async () => {  
  console.log("Veri çekiliyor...");  
  const response = await fetch("https://jsonplaceholder.typicode.com/todos/1");  
  const data = await response.json();  
  console.log("Veri alındı:", data);  
};  
getData();
```

Web Workers Nedir?

- 📌 Ana thread'i rahatlatan bir teknoloji
- 📌 Ağır işlemleri arka planda çalıştırarak sayfanın donmasını önler

Kullanım Alanları:

- Büyük veri işlemleri (örneğin, resim işleme)
- Yoğun hesaplama gerektiren işlemler

Web Workers ile Büyük Veri İşleme

worker.js

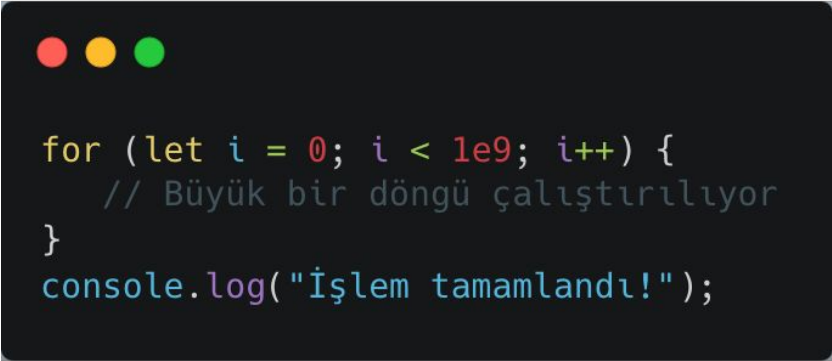
```
self.onmessage = function(event) {  
    let result = event.data * 2;  
    postMessage(result);  
};
```

main.js

```
const worker = new Worker("worker.js");  
worker.postMessage(5);  
worker.onmessage = function(event) {  
    console.log("Sonuç:", event.data);  
};
```

Web Workers Neden UI'ye Yük Bindirmiyor?

JavaScript tek iş parçacıklı (single-threaded) bir dildir. Bu nedenle, CPU'yu yoğun kullanan bir işlem yapılırsa, sayfa yanıt vermez hale gelebilir.



```
for (let i = 0; i < 1e9; i++) {  
    // Büyük bir döngü çalıştırılıyor  
}  
console.log("İşlem tamamlandı!");
```

Bu kod **UI'yi kilitleyecek** ve sayfa yanıt vermeyecektir. **Butonlara tıklanamaz, sayfa kaydırılamaz.**

Web Workers, UI Thread'ini Nasıl Korur?

Web Workers **ana thread (main thread)** yerine **arka planda (background thread)** çalışır. Böylece, **ağır işlemleri asenkron olarak çalıştırabilir ve UI'yi rahatlatabiliriz.**

Ana Thread ve Worker Thread'in Çalışma Mantiği

- **Ana Thread** = HTML, CSS, DOM işlemleri (Kullanıcı arayüzü)
- **Worker Thread** = Arka planda ağır işlemler

Web Workers'in Faydaları

- ✓ Ana Thread'i rahatlatır → UI'nin donmasını önler
- ✓ Büyük veri işlemlerini arka planda yapar → Kullanıcı deneyimi iyileşir
- ✓ Ağır hesaplamaları UI'ye etki etmeden çalıştırır
- ✓ Asenkron olduğu için performansı artırır

Bazı Kaynaklar

[https://developer.mozilla.org/en-US/docs/Web/API/Web Workers API/Using web workers](https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers)

<https://www.freecodecamp.org/news/javascript-async-await-tutorial-learn-callbacks-promises-async-await-by-making-icecream/>

Ödev

Bir HTML dosyasına 'ins-api-users' sınıfına sahip bir eleman (div) ekleyin. Aşağıdaki işlemlerin hepsini JS dosyası içerisinde bu elemana append olacak şekilde (CSS dahil) yönetin.

API'den Kullanıcı Verisini Çekme

Fetch API kullanarak <https://jsonplaceholder.typicode.com/users> adresinden kullanıcı verisini çekin.

Gelen veriyi bir Promise içinde yönetin.

Hata yönetimi yapın (örneğin, API'ye ulaşılamazsa kullanıcıya hata mesajı gösterin).

localStorage Kullanımı

Çekilen veriyi localStorage'a kaydedin (1 gün süreyle)

Eğer sayfa yenilendiğinde localStorage'da veri varsa, tekrar API'den veri çekmeden bunu kullanın.

Basit Bir Arayüz Oluşturma

Kullanıcıları listeleyen bir HTML sayfası oluşturun.

Kullanıcı adı, e-posta ve adres bilgilerini görüntüleyin.

Kullanıcıları **silme** seçeneği ekleyin (silinen kullanıcı localStorage'dan da kaldırılmalı). //Opsiyonel